

Assignment 4:

**Heap Data Structures: Implementation,
Analysis and Applications**

Shreeya Gadgil

For MSCS 532 – Algorithms and Data Structures

A) Heap Sort Implementation and Analysis:

a. Time complexity Analysis:

- i. Methodology: The algorithm operates in two distinct phases. First, it transforms the input array into a Max-Heap while processing elements from the leaves to the root, ensuring that every parent node is greater than or equal to its children. Next, it loops through the array, swaps the top maximum value with the last unsorted element at the end, while shrinking the active tree size by one. Then it runs a restoration step called “heapify” to push the new top element down to its correct position. By repeating this cycle, Heapsort builds a perfectly sorted array from right to left without using extra memory.

ii. Mathematical analysis:

For all cases, the initial max-heap creation step takes $O(n)$ time. This happens because the work needed decreases exponentially as we descend the tree structure.

$$\text{Time taken to build a Heap} = \sum_{i=0}^{\lfloor \log(n) \rfloor} \left(\frac{n}{2^{i+1}} \right) \cdot O(i) = O(n)$$

This time would add directly to all cases since it is a required step.

- **Worst Case:**

This would happen when the input has unique values arranged so that every time we extract the root, the new swapped element would sift down to the bottom of the tree.

The sort loop runs $n-1$ times. At each step k , the element must sift down the entire tree which would be $\log_2 k$ steps.

$$\text{Time for sort loop} = \left[\sum_{k=1}^{n-1} \log_2 k \right] = \log_2((n-1)!)$$

After applying Stirling's approximation, we get

$$\text{Total time} = O(n) + O(n \log n) = O(n \log n)$$

Here, Stirling's approximation is $\log_2(n!) \approx n \log_2 n - n \log_2 e + O(\log_2 n)$,

- **Average Case:**

This would happen when the input is a random permutation of distinct elements. When you swap the bottommost leaf to the root, the element is likely to be very small. On average, it still travels through some significant fraction of the tree's height. The depth of the sift down remains proportional to $\log n$ for most of the extractions, the summation looks like

$$\text{Total time} = O(n) + \sum_{k=1}^{n-1} (\log_2 k - c) = \Theta(n \log n)$$

Note: here, c is the average number of levels a node stops short of the very bottom level during the sift-down process.

- **Best Case:**

The best-case scenario would be for an already sorted array. Even if the array is sorted, Heapsort will still break down the heap structure. Swapping elements forces small leaf nodes to the root, which must sift right back down to the bottom. Therefore,

$$\text{Time for sort loop} = \left[\sum_{k=2}^n \log_2 k - c \right] = \log_2(n!) - \sum_{k=2}^n c$$

Applying Stirling's approximation here ($\log_2(n!) = n \log_2 n - n \log_2 e + O(\log_2 n)$), this would evolve to

$$\text{Time for sort loop} = \Omega(n \log n) - \Omega(n)$$

Therefore,

$$\text{Total time} = \Omega(n) + \Omega(n \log n) - \Omega(n) = \Omega(n \log n)$$

b. Space Complexity

Heapsort is an in-place sorting algorithm. Unlike MergeSort, it does not need dynamic array allocations. During iterative implementations, the auxiliary memory used is $O(1)$, which requires managing a few variables for indices and pointers.

c. Additional Overheads

When heapify uses recursion, the function has to call itself multiple times to move down the tree. Each of these calls takes up some memory on the computer's execution cell stack to keep track of where it is. Since the maximum number of steps is equivalent to the tree's height, the stack space consumption goes to $O(\log n)$. This overhead can be eliminated by rewriting the recursive function as an iterative while loop, which results in $O(1)$ space.

Apart from this, Heapsort relies on parent-child pointers mapped from i to $2i+1$ indices. As the algorithm runs, it jumps back and forth across different parts of the memory. This ruins CPU cache locality. L1/L2 caches work best when a program reads data stored side by side. But Heapsort forces the computer to pause and fetch data from the much slower RAM. This causes a lot of internal physical delay, and makes Heapsort slower in testing.

d. Run-time Comparison of Heapsort with Quicksort and Mergesort

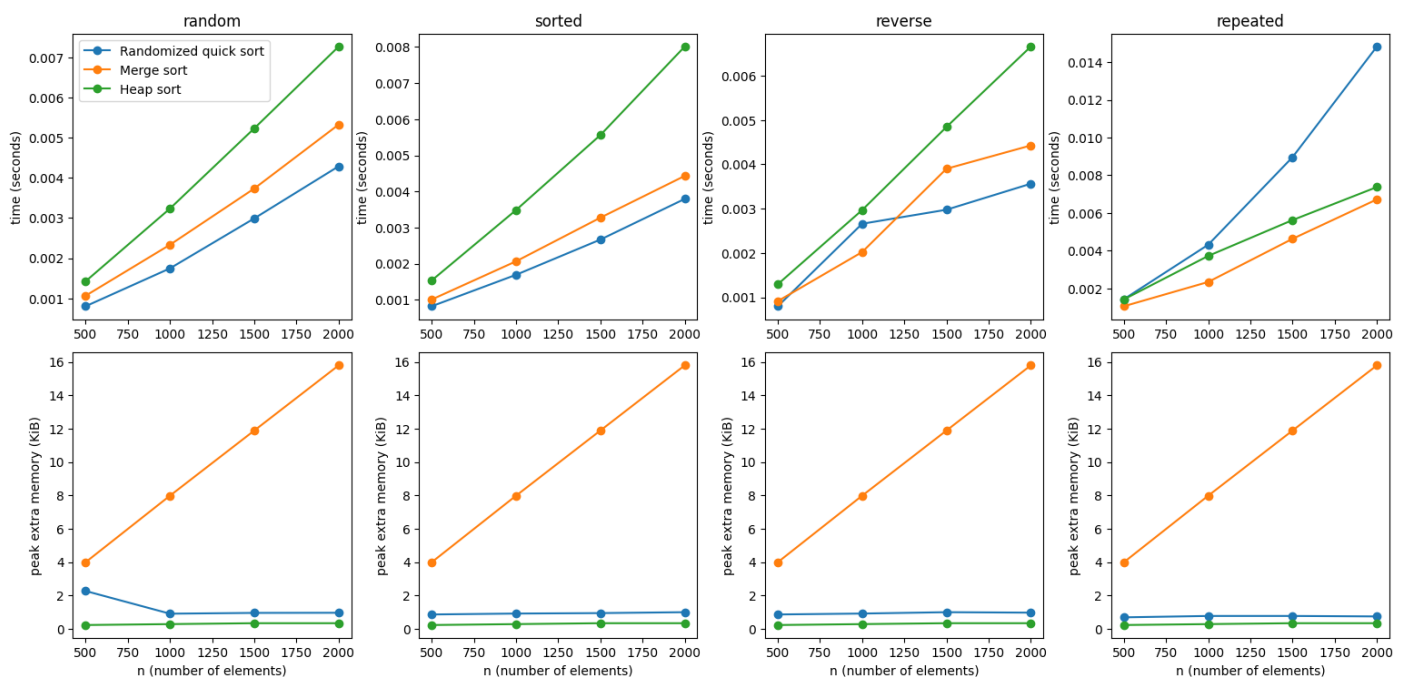
The empirical results broadly confirm the theoretical time complexity of all three algorithms while also revealing the extent to which constant factors and cache behavior affect real-world performance. Across the random, sorted, and reverse-sorted distributions, randomized quicksort consistently ran fastest, followed by merge sort, with heap sort the slowest of the three. This ordering does not contradict the theoretical analysis, since all three algorithms share the same $O(n \log n)$ growth rate on these distributions; rather, it reflects differences in constant overhead. Quicksort benefits from in-place partitioning and strong cache locality, since it operates on contiguous memory with a simple comparison-and-swap pattern. Heap sort, by contrast, repeatedly accesses parent and child indices (i , $2i+1$, $2i+2$) that are often far apart in memory, which disrupts CPU cache performance and adds real-world overhead not captured by Big-O notation alone.

The repeated-values distribution produced the most significant result in the dataset. Here, randomized quicksort's advantage disappeared entirely, and it became the slowest algorithm by $n = 2000$, while heap sort and merge sort were largely unaffected by the change in input distribution. This result directly illustrates quicksort's known worst-case

vulnerability: when the input contains many duplicate keys, partitioning repeatedly produces unbalanced splits, pushing performance toward $O(n^2)$ rather than the expected $O(n \log n)$. Heap sort's relative insensitivity to this distribution empirically confirms its guaranteed $O(n \log n)$ worst-case bound, since its performance does not depend on the structural properties or repetition of the input values, only on the number of elements being sorted.

The memory results align closely with the theoretical space complexity of each algorithm. Merge sort's peak auxiliary memory usage grew linearly with input size across every distribution, consistent with its $O(n)$ requirement for temporary arrays during merging. Heap sort's memory usage remained close to zero and effectively constant regardless of input size, matching its in-place, $O(1)$ auxiliary space design. Quicksort fell in between, with a small and roughly constant memory footprint consistent with its expected $O(\log n)$ recursion depth.

Taken together, these results reinforce a central engineering trade-off: quicksort offers the best average-case performance but is vulnerable to specific input patterns, merge sort guarantees predictable performance at the cost of significant memory overhead, and heap sort sacrifices average-case speed and cache efficiency in exchange for a strict $O(n \log n)$ worst-case guarantee and minimal memory usage. The choice between these algorithms in practice therefore depends not only on expected input size, but also on the structure of the data and whether a worst-case performance guarantee is more valuable than typical-case speed.



B) Priority Queue Implementation and Analysis

a. Data Structure choice Explanation:

The priority queue was implemented using an array-based heap, represented internally as a Python list. This choice was made over node-based tree structure for reasons given below.

First of all, an array representation allows parent-child relationships to be computed directly using the index rather than being stored as pointers. For a node at index i , its parent is located at $(i-1)/2$, its left and right children at $2i+1$ and $2i+2$ respectively. This removes the memory overhead of storing references on every node, thus simplifying operations.

Second, array based heaps offer better cache locality. Because the elements of a list are stored contiguously in memory, traversing across nearby heap positions is beneficial than having a pointer-based tree structure.

A min heap was chosen over a max heap in this case, where smaller numeric priority values are considered as more urgent. This choice means that an Earliest-First-Deadline approach can be used. By setting a task's priority equal to its deadline, the task with the nearest deadline is always the one with the smallest priority numerically. This ensures that it is always on top/at the root of the heap.

b. Core Operations:

The priority queue supports the following operations:

- **Insert(task):** A new task is appended at the end of the array, and then repeatedly swapped with its parent until the min-heap property is restored.
Time complexity: Appending to the end of the list is $O(1)$ and the sifting up operation can go through swaps equal to the height of the heap, which is $O(\log n)$ for a binary heap of n elements. Hence the total time complexity would be $O(\log n)$.
- **extract_min():** This operation removes and returns the task at the root of the heap. The last element in the array is moved to the root position and then sifted down till the heap is restored.
Time complexity: $O(\log n)$. Removing the last element is $O(1)$, and the sift down operation would go through swaps equal to the height of the heap, which is $O(\log n)$. This takes the total time complexity to $O(\log n)$.
- **increase_key/decrease_key:** these operations can modify the priority of a task already existing in the queue. **decrease_key** lowers the task's priority value, making it more urgent, and **increase_key** increases the task's priority value, making it less urgent, and shifts the task downwards if needed.
Time complexity: The PriorityQueue class maintains a separate dictionary, mapping every task id to its current index in the heap array. Without this, locating a task would take $O(n)$ time to scan the array. With the position map, kept in sync by the **swap()** function, the lookup is $O(1)$. The sift down operation takes $O(\log n)$, bringing the total complexity to $O(\log n)$.

- `Is_empty()`: This function checks if there are any tasks in the queue or not.

Time complexity: $O(1)$

c. Implementation:

The implementation separates internal heap-maintenance logic from the actual operations, which keeps the heap logic reusable and easy to test. Edge cases are explicitly handled: calling `extract_min()` on an empty queue raises an `IndexError`, and calling `increase_key/decrease_key` on a non-existent task, or with a value that moves priority in the wrong direction, raises an appropriate exception rather than corrupting the heap. To validate all four operations, a demonstration script was written that performs each operation in sequence to validate basic functionality, before putting it all into the scheduler simulation.

d. Scheduler simulation and Analysis:

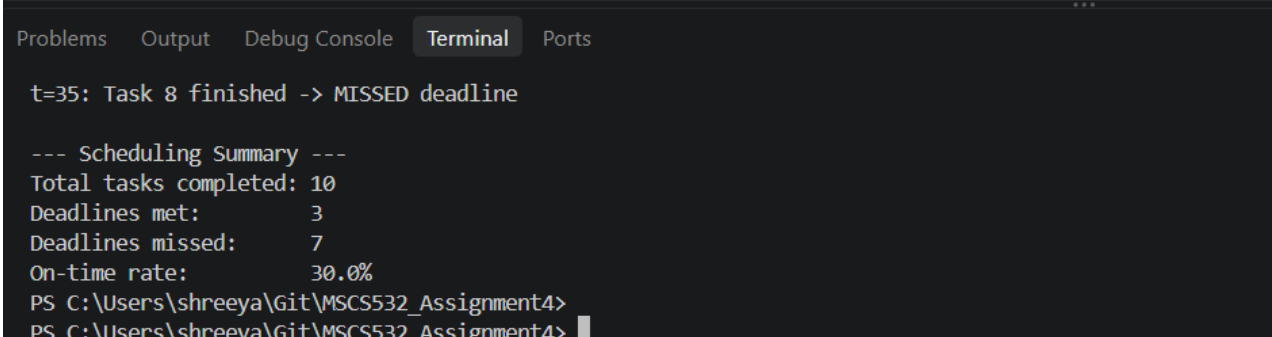
To demonstrate the priority queue in simulation, an Earliest-Deadline-First(EDF) scheduler simulation was built on top. Tasks are generated with arrival times, execution durations and deadlines all random. As time advances, tasks are inserted into the priority queue once their arrival time has passed, and whenever the processor becomes free, the task with the nearest deadline is removed and executed. The simulation keeps a track of when each task was completed.

In one simulation run, the scheduler completed all tasks but met only 3 of 10 deadlines(a 30% on time rate).

```

8 def generate_tasks(n, max_arrival=20, max_duration=5, max_deadline_offset=15, seed=None):
    ....

```



```

t=35: Task 8 finished -> MISSED deadline

--- Scheduling Summary ---
Total tasks completed: 10
Deadlines met:         3
Deadlines missed:      7
On-time rate:          30.0%
PS C:\Users\shreeya\Git\MSCS532_Assignment4>
PS C:\Users\shreeya\Git\MSCS532_Assignment4>

```

Examining the trace, it was observed that the first three tasks all completed well before their deadlines. During this period, the processor had idle time between tasks. But later in the simulation, six additional tasks arrived in rapid succession while the processor was still working on the earlier backlog. From this point further, nearly every task missed its deadline, including tasks whose deadlines weren't tight. They just weren't started because the processor remained occupied with earlier tasks.

On another simulation with relatively relaxed parameters, the scheduler met 90% of the deadlines.

```
8 def generate_tasks(n, max_arrival=10, max_duration=2, max_deadline_offset=15, seed=None):
    """
    """
```

Problems Output Debug Console **Terminal** Ports

```
--- Scheduling Summary ---
Total tasks completed: 10
Deadlines met:          9
Deadlines missed:       1
On-time rate:           90.0%
```

On a single processor, EDF is optimal in the sense that if any scheduling order can meet all task deadlines, EDF will find it. This is applicable only when the system is not overloaded. Once the rate of incoming work exceeds the processors capacity to keep up, missed deadlines are unavoidable regardless of which scheduling algorithm is used. This happens even if the processor is working efficiently and making optimal decisions at every step. Hence scheduling quality in practice depends not only on the algorithm's theoretical guarantees, but also on the system load and processing capabilities, regardless of a well-designed priority queue.

Repository: https://github.com/shreeya-up/MSCS532_Assignment4.git